



NON SEQUITUR PUBLISHING - WHITE PAPERS

AI Governance Series - Paper 3

Agile Scrum, Agentics, and the Skipjack Protocol: Governed Collaboration Between Human Judgment and Agentic Execution

Justin H. Kuiper, CISSP

ORCID 0009-0008-7099-3286

Cover revision 2026-09-15 | Manuscript unchanged

Abstract

Agile Scrum has governed software development execution for more than two decades. Its core commitments — iterative delivery, transparency, team collaboration, and responsiveness to change — remain valid in the age of AI. What Scrum was not designed for is a world in which artificial intelligence agents actively participate in execution: not as tools that human team members use, but as actors that decompose, execute, iterate, and report within the same workflow as their human counterparts.

This paper bridges that gap. It extends traditional Agile Scrum into an agentic operating model — a framework in which human teams and AI systems function as a unified execution layer, governed by a shared protocol. That protocol is the Skipjack Protocol.

The extension works by remap, not replacement. The Product Owner maps to a Governance Layer of humans with domain authority who define intent, constraints, and prioritization. The Scrum Master maps to an Orchestrator responsible for task routing, flow control, and system health. The Development Team maps to an Agentic Execution Layer of AI agents and human operators who execute within governed boundaries. The Backlog maps to persistent memory and task queues, maintaining epistemic continuity across sprint cycles.

The Skipjack Protocol governs this system through four enforcement mechanisms: context integrity and epistemic continuity; structured task decomposition and routing; human-in-the-loop control points; and iterative feedback loops that close between human judgment and agentic execution. Together, they ensure that the three-tier architecture operates with integrity at any scale.

Paper Five is the fifth paper in the Mission-Ready AI pentalogy. The key to scaling AI is not autonomy alone — it is governed collaboration between human judgment and agentic execution.

How to cite

Justin H. Kuiper, CISSP (2026). *Agile Scrum, Agentics, and the Skipjack Protocol: Governed Collaboration Between Human Judgment and Agentic Execution* (v1.0-preprint). Non Sequitur Publishing.
<https://nonsequitur.tech/white-papers/skipjack-protocol/>

© 2026 Non Sequitur Publishing. All rights reserved. Citation with full attribution is permitted. Reproduction, redistribution, derivative works, and use as input to machine-learning training are **not** permitted without written permission. See <https://nonsequitur.tech/citation-policy/>.
Canonical URL: <https://nonsequitur.tech/white-papers/skipjack-protocol/>

ABSTRACT

Agile Scrum has governed software development execution for more than two decades. Its core commitments — iterative delivery, transparency, team collaboration, and responsiveness to change — remain as valid in the age of AI as they were when Ken Schwaber and Jeff Sutherland first formalized them.¹ What Scrum was not designed for is a world in which artificial intelligence agents actively participate in execution: not as tools that human team members use, but as actors that decompose, execute, iterate, and report within the same workflow as their human counterparts.

This paper bridges that gap. It extends traditional Agile Scrum into an *agentic operating model* — a framework in which human teams and AI systems function as a unified execution layer, governed by a shared protocol. That protocol is the *Skipjack Protocol*.

The extension works by remap, not replacement. The authority relationships that Scrum encodes — who defines intent, who manages flow, who executes — are preserved. What changes is who holds those roles. The Product Owner maps to a Governance Layer: humans with domain authority who define intent, constraints, and prioritization. The Scrum Master maps to an Orchestrator: the entity responsible for task routing, flow control, and system health. The Development Team maps to an Agentic Execution Layer: a hybrid of AI agents and human operators who execute tasks within governed boundaries. The Backlog maps to persistent memory and task queues, maintaining epistemic continuity across sprint cycles.

The Skipjack Protocol governs this system through four enforcement mechanisms: context integrity and epistemic continuity; structured task decomposition and routing; human-in-the-loop control points; and iterative feedback loops that close between human judgment and agentic execution. Together, they ensure that the three-tier architecture operates with integrity at any scale.

Paper Five is the fifth paper in the Mission-Ready AI pentalogy. Paper Four (Alistair Prime in a Box) defined the self-sufficient cognitive node — the DDIL-tolerant execution unit that Skipjack operates over.² Paper Five defines how humans and agents operate across those nodes. The pentalogy's argument completes here.

The key to scaling AI is not autonomy alone. It is governed collaboration between human judgment and agentic execution.

1. AGILE SCRUM MEETS AGENTICS: THE GAP AND THE NEED

1.1 AGILE SCRUM'S CORE COMMITMENTS

The Scrum Guide's definition of Scrum has remained remarkably stable since its 2020 revision: a lightweight framework that helps people, teams, and organizations generate value

1 Ken Schwaber and Mike Beedle, *Agile Software Development with Scrum* (Upper Saddle River, NJ: Prentice Hall, 2001); Ken Schwaber, *Agile Project Management with Scrum* (Redmond, WA: Microsoft Press, 2004).

2 Justin H. Kuiper, CISSP, "Alistair Prime in a Box: Designing Self-Sufficient Cognitive Systems at the Edge," Non Sequitur Publishing, 2026, v1.0-preprint, <https://nonsequitur.tech/white-papers/alistair-prime-in-a-box/>.

through adaptive solutions for complex problems.³ The underlying commitments — empirical process control, iterative delivery, self-organizing teams, continuous improvement — predate the current formulation and have proven durable across a wide range of organizational contexts.

Three commitments are load-bearing for the analysis in this paper:

Iterative delivery. Work is organized into fixed-length sprints. At the end of each sprint, a potentially shippable increment is produced and reviewed. This structure creates regular cadences for human review, course correction, and re-prioritization.

Transparency. All aspects of the process — the backlog, the sprint goal, the current work, the impediments — are visible to all stakeholders. This visibility is the precondition for effective inspection and adaptation.

Defined role structure. Scrum defines three roles with distinct authority and responsibility: the Product Owner (who owns the backlog and defines priorities), the Scrum Master (who facilitates the process and removes impediments), and the Development Team (who execute the work). These roles encode an authority structure, not merely a division of labor.

These commitments are not challenged by the agentic model. They are, if anything, strengthened by it: iterative delivery provides the cadence for human-AI feedback loops; transparency is a prerequisite for meaningful oversight of agentic execution; defined role structure is the foundation on which the role remap in §2 is built.

1.2 THE AGENTIC PARTICIPATION GAP

Current AI integration in Agile environments follows a consistent pattern: AI is a tool that team members use. A developer uses an AI coding assistant. A product manager uses an AI to draft user stories. A QA engineer uses an AI to generate test cases. In each case, the AI responds to a human prompt, the human evaluates the output, and the human decides what to do with it. The AI participates as a sophisticated instrument, not as an actor in the process.

The emerging operational reality is different. AI agents in current and near-term deployments do not merely respond to prompts. They decompose objectives into sub-tasks, execute those tasks, evaluate their own outputs, iterate on failure, coordinate with other agents, and surface completion or escalation signals. They participate in execution in a way that current Scrum has no model for. The agent is not a team member's tool. The agent is doing the work.

This creates a participation gap: a mismatch between what agents can do and what the framework governing their operation was designed to handle. The gap has two edges.

On one side, Scrum's role structure does not account for an actor that executes without requiring instruction at each step. The Scrum Master facilitates a human team; the Development Team self-organizes within a sprint. Neither model encompasses an agent that decomposes and executes a task queue while the human team attends to other concerns. There is no defined point at which the Scrum Master routes a task to an AI agent, no defined way for the agent to signal that it has encountered a decision outside its authorized scope, no defined ceremony at which the human team reviews what the agents produced.

On the other side, the absence of a governance model for agentic participation creates risk. Without defined roles, without structured task decomposition, without HITL control points, and without feedback loops that close between human judgment and agent execution, two failure modes appear. In the first, agents operate beyond sanctioned boundaries — making decisions

³ Ken Schwaber and Jeff Sutherland, "The Scrum Guide," Scrum.org, 2020, <https://scrumguides.org/scrum-guide.html>.

that belong to the Product Owner, executing work that has not been authorized, producing outputs that no human has reviewed before they enter the system of record. In the second, every agent action requires manual human approval — the governance overhead eliminates the efficiency benefit of agentic execution entirely. Neither failure mode is acceptable. The Skipjack Protocol targets the space between them.

1.3 WHY EXTENSION, NOT REPLACEMENT

The framing of this paper is deliberate: Skipjack extends Scrum, it does not replace it. This is not a diplomatic hedge. It reflects a substantive judgment.

Scrum's core commitments — iterative delivery, transparency, defined roles, inspection and adaptation — are not artifacts of a pre-AI era that must be discarded. They are solutions to enduring coordination problems that become *more* important, not less, when AI agents participate in execution. Iterative delivery creates the cadence for human-AI feedback. Transparency is a prerequisite for meaningful oversight. Defined roles prevent authority conflicts. Inspection and adaptation allow the operating model to improve as the human-AI team accumulates experience.

What Scrum lacks is not a philosophy but a model: a concrete account of how agentic actors fit into the role structure, how tasks are routed to them, how their outputs are reviewed, and how the feedback from that review propagates back into their operation. The Skipjack Protocol provides that model. It is not a new framework layered on top of Scrum. It is the extension of Scrum's existing logic into the agentic domain.

1.4 PAPER FIVE'S POSITION IN THE PENTAGLOGY

Paper Four of this series defined the self-sufficient cognitive node: a bounded, deployable cognitive unit that operates under DDIL conditions, governed by pre-authorized human intent, with five pillars — local inference, persistent memory, agentic orchestration, governance enforcement, and execution capability.⁴ The node is the *unit* of agentic execution in the system Paper Five describes.

Paper Five describes how humans and agents operate *across* those nodes. The Skipjack Protocol is the operating model. The role remap is the structure. The four enforcement mechanisms are the protocol's teeth. Together, they answer the question that Paper Four's node makes possible: now that we have a DDIL-tolerant, governance-embedded cognitive unit, how do we operate one — or many — of them alongside human teams?

2. THE ROLE REMAP: SCRUM TO SKIPJACK ARCHITECTURE

2.1 WHY A REMAP, NOT A REPLACEMENT

The Scrum role model works because it encodes real authority relationships. The Product Owner holds authority over what gets built and in what order. The Scrum Master holds authority over how the process runs and what impedes it. The Development Team holds authority over how the work is executed within the sprint. These authority relationships are not arbitrary conventions; they reflect the structure of accountability in a product development organization.

⁴ Justin H. Kuiper, CISSP, "Alistair Prime in a Box: Designing Self-Sufficient Cognitive Systems at the Edge," Non Sequitur Publishing, 2026, v1.0-preprint, <https://nonsequitur.tech/white-papers/alistair-prime-in-a-box/>.

In an agentic operating model, these authority relationships do not disappear. They persist — but the *holders* of the roles change. An AI agent is not the appropriate holder of Product Owner authority: it should not set priorities, define intent, or determine what the organization builds. An AI agent may be an appropriate participant in the Orchestrator function, routing tasks and monitoring system health, under human governance. AI agents are appropriate members of the Agentic Execution Layer, executing tasks within the scope they have been assigned. The authority structure of Scrum maps onto the human-AI system; the question is which actors hold which roles.

The remap below formalizes this. It is not a suggestion. Each row reflects a structural claim about where human authority must be located in an AI-augmented execution system.

2.2 THE ROLE REMAP TABLE

Traditional Scrum
Agentic remap
Product Owner
Governance Layer
Scrum Master
Orchestrator
Development Team
Agentic Execution Layer
Backlog
Persistent memory + task queues
Each row is analyzed in depth in the sections that follow.

2.3 GOVERNANCE LAYER — PRODUCT OWNER REMAP

The Product Owner in Scrum is the single point of accountability for the product backlog. They define what gets built, in what order, and why. They are the voice of the stakeholders within the Scrum team, the person who says yes or no to features, and the authority that keeps the team oriented toward value delivery.

In the agentic remap, this authority maps to the *Governance Layer*. The Governance Layer defines intent, constraints, and prioritization. It is the source from which all authorized activity flows. It sets the boundaries within which the Orchestrator may route tasks and within which the

Agentic Execution Layer may execute. Nothing enters the execution pipeline that the Governance Layer has not sanctioned.

The Governance Layer is composed of humans. This is not a design preference — it is a structural requirement. The authority to define intent and set constraints is the authority that determines what the system does and does not do. Delegating that authority to an AI is not a matter of technical capability; it is a matter of accountability. The Governance Layer must be composed of humans who can be held accountable for the intent they define, the constraints they establish, and the outcomes they authorize.⁵

The Governance Layer is not passive. It does not merely approve a backlog at sprint planning and then disengage until review. It defines the pre-authorized decision envelopes within which execution operates — a concept introduced in Paper Four's governance pillar⁶ — and revisits those envelopes at defined intervals. It reviews execution outputs at sprint review. It participates in the retrospective feedback loop. It is active throughout the sprint, even when it is not intervening in execution step by step.

2.4 ORCHESTRATOR — SCRUM MASTER REMAP

The Scrum Master in Scrum is responsible for process integrity: facilitating ceremonies, removing impediments, ensuring that the team follows the agreed process, and coaching the organization on Scrum adoption. The Scrum Master does not direct the work — that is the Development Team's authority — but they ensure the conditions under which the work can be done effectively.

In the agentic remap, this function maps to the *Orchestrator*. The Orchestrator is responsible for task routing, flow control, system health monitoring, and escalation management. It decomposes the objectives defined by the Governance Layer into executable task specifications and routes those specifications to the appropriate members of the Agentic Execution Layer. It monitors execution health — which tasks are progressing, which are blocked, which have surfaced a scope question that requires Governance Layer attention. When a task approaches the boundary of the pre-authorized envelope, the Orchestrator escalates to the Governance Layer.

The Orchestrator may be human, AI, or hybrid. A human Orchestrator brings contextual judgment and flexibility; an AI Orchestrator brings speed, consistency, and the ability to monitor many parallel execution threads simultaneously. In practice, the Orchestrator function in the near term will often be hybrid: an AI system that handles high-volume, well-defined routing decisions, with human oversight for novel situations, escalations, and governance interactions. What the Orchestrator is not is autonomous: its task routing decisions are bounded by the Governance Layer's intent parameters. It does not set direction; it translates direction into execution.

5 Justin H. Kuiper, CISSP, "Mitigating Confident Misalignment in Agentic Systems: The HGC³AE² Framework," Non Sequitur Publishing, 2026, v1.0-preprint, <https://nonsequitur.tech/white-papers/hgc3ae2/>; Jonas Schuett, Anka Reuel, and Alexis Carlier, "How to Design an AI Ethics Board," *AI and Ethics* 5, no. 2 (2024): 863–881, <https://doi.org/10.1007/s43681-023-00409-y>.

6 Kuiper, "Alistair Prime in a Box," §2.5 (Governance and Policy Enforcement) and §4.3 (Decision Envelopes and Scope Boundaries).

2.5 AGENTIC EXECUTION LAYER — DEVELOPMENT TEAM REMAP

The Development Team in Scrum is the group of professionals who execute the work. They are self-organizing — they determine how to accomplish the sprint goal — and cross-functional — they collectively possess the skills needed to deliver the increment. They commit to a sprint plan and are accountable for delivery against it.

In the agentic remap, this maps to the *Agentic Execution Layer*: a composition of AI agents, human operators, or hybrid teams who execute the tasks assigned by the Orchestrator. The Agentic Execution Layer executes within the scope it has been given. It does not set its own objectives, does not extend its own decision envelope, and does not make scope judgments that belong to the Orchestrator or the Governance Layer.

Agents in the Agentic Execution Layer report execution status to the Orchestrator. When a task proceeds within defined parameters, the agent executes and reports completion. When a task encounters a condition that appears to approach the boundary of the pre-authorized envelope, the agent surfaces an escalation signal to the Orchestrator — not a decision, a signal. The decision of whether to proceed, halt, or escalate further belongs to the Orchestrator (within its own authority) or to the Governance Layer (for decisions that exceed the Orchestrator's mandate).

The self-organizing property of Scrum's Development Team translates here with a critical qualification: self-organization in the Agentic Execution Layer is self-organization *within defined scope*, not self-direction. An agent that can determine the most efficient way to accomplish a well-defined task is valuable. An agent that redefines the task because the original definition seemed suboptimal has exceeded its role.

2.6 PERSISTENT MEMORY AND TASK QUEUES — BACKLOG REMAP

The Backlog in Scrum is the ordered list of work items that the Product Owner maintains. It is prioritized (the most important work is at the top), refined (backlog items are elaborated as they approach the sprint boundary), and dynamic (the backlog changes as the Product Owner's understanding of priorities evolves). Sprint planning draws from the top of the backlog to populate the sprint.

In the agentic remap, the Backlog becomes *persistent memory and task queues*. This is not merely a relabeling. The Backlog in Scrum is a list of features to be built; the persistent memory and task queue system is an epistemically continuous record of everything the system has done, is doing, and intends to do, together with the context that informs those intentions.

Persistent memory — introduced in Paper Four's Pillar 2⁷ — preserves the accumulated context of prior execution cycles. The task queue translates the Governance Layer's current intent into execution-ready work items, ordered by the Orchestrator. Sprint planning in the agentic model is the process by which the Governance Layer updates the intent that populates the task queue. Backlog refinement is the process by which the Orchestrator translates Governance Layer intent into task specifications granular enough for Agentic Execution Layer agents to execute.

The continuity property is critical. In Scrum, the backlog carries forward across sprints — items not completed in Sprint N remain in the backlog for Sprint N+1. In the agentic model, persistent memory ensures not just that unfinished tasks carry forward, but that the *context* of those tasks — why they were initiated, what the Governance Layer's intent was, what was

7 Kuiper, "Alistair Prime in a Box," §2.3 (Persistent Memory).

learned during prior execution attempts — carries forward with them. This is the architectural response to the epistemic degradation problem Paper Two identified.⁸

2.7 SCRUM CEREMONIES REMAPPED

The ceremony structure of Scrum translates directly into the Skipjack operating cycle:

Sprint planning becomes the cycle in which the Governance Layer defines intent and the Orchestrator performs initial task decomposition. The output is a task queue populated with Governance Layer-authorized, Orchestrator-specified work items.

Daily standup becomes the Orchestrator's flow-control check: what tasks completed since the last checkpoint, what is in progress, what has surfaced an escalation signal. This may run continuously for AI-executed tasks rather than at a fixed daily cadence.

Sprint review becomes the primary HITL control point: the Governance Layer reviews execution outputs against the intent defined at sprint planning. Outputs that satisfy the original intent are accepted into the system of record. Outputs that deviate prompt a governance decision about whether the deviation was acceptable, what caused it, and whether the task specification or decision envelope needs revision.

Sprint retrospective becomes the feedback integration cycle: the Governance Layer and Orchestrator together decide what the sprint review revealed about the persistent memory schema, the task routing logic, and the pre-authorized decision envelopes. Approved feedback updates the system. The updated system informs the next sprint planning cycle.

The ceremonies are the heartbeat of the governance model. They are not efficiency overhead. They are the mechanism through which human judgment continuously informs and corrects agentic execution.

The ceremony remap described above rests on a foundation of peer-reviewed Scrum research. The mapping is not an analogy drawn informally from practitioner experience — it is an extension of an authority structure that the empirical Scrum literature has studied extensively. Hron and Obwegeser's systematic review of Scrum adaptation in practice (2022)⁹ documents the range of ways Scrum's ceremony and role structure has been modified across organizational contexts, establishing that the core authority model is robust across adaptation while the ceremony specifics are adjustable. The Skipjack ceremony remap is a structured adaptation within this tradition.

Sassa et al.'s systematic literature review of Scrum (2023)¹⁰ synthesizes the empirical record of Scrum adoption, outcomes, and common adaptation points. Pacagnella et al.'s structural equation modelling analysis of Agile critical success factors (*IEEE Transactions on*

8 Justin H. Kuiper, CISSP, "Epistemic Constraints and Semantic Compression in Natural Language Processing: A Theoretical Foundation for the HGC³AE² Framework," Non Sequitur Publishing, 2026, v1.0-preprint, <https://nonsequitur.tech/white-papers/epistemic-constraints/>.

9 Martin Hron and Nikolaus Obwegeser, "Why and how is Scrum being adapted in practice: A systematic review," *Journal of Systems and Software* 183 (2022): 111110, <https://doi.org/10.1016/j.jss.2021.111110>.

10 Adrielle Cristina Sassa, Isabela Alves de Almeida, Tábata Nakagomi Fernandes Pereira, and Milena Silva de Oliveira, "Scrum: A Systematic Literature Review," *International Journal of Advanced Computer Science and Applications* 14, no. 4 (2023), <https://doi.org/10.14569/IJACSA.2023.0140420>.

Engineering Management, 2024)¹¹ identifies the specific Scrum authority elements — Product Owner domain authority, Scrum Master facilitation independence, team-level execution autonomy — that predict project success. The Skipjack remap preserves the three authority elements that Pacagnella et al. identify as load-bearing: the Governance Layer carries the Product Owner’s domain authority (setting intent, accepting or rejecting outputs); the Orchestrator carries the Scrum Master’s facilitation role (enabling execution without directing it); and the Agentic Execution Layer carries the team’s execution autonomy (within the authorized scope).

No current agent harness framework draws on Agile governance theory to ground its role model. AutoGen, LangGraph, CrewAI, and MetaGPT define their role taxonomies from computational architecture requirements — what roles a multi-agent system needs to execute tasks — rather than from the governance and accountability literature that Scrum encodes. The Skipjack role model is not a re-invention of the wheel; it is the peer-reviewed wheel applied to a problem the frameworks have not addressed.

3. THE SKIPJACK PROTOCOL: FOUR ENFORCEMENT MECHANISMS

3.1 WHAT THE SKIPJACK PROTOCOL IS

The Skipjack Protocol is a governing structure. It is not a software framework — it does not specify a particular programming language, API surface, or toolchain. It is not a certification — there is no compliance checklist that a system can clear once and then operate without further governance attention. It is not a process methodology competing with Agile or replacing Scrum. It is the set of enforcement mechanisms that ensure the three-tier role model defined in §2 operates with integrity at scale.

Paper Three of this series introduced the Skipjack Protocol as an application-layer response to the epistemic integrity requirements established in Papers One and Two.¹² Paper Five formalizes it. The formalization has two parts: the role remap (§2) defines the authority structure; the enforcement mechanisms (§3–§4) define how that authority is maintained in practice, across task boundaries, sprint cycles, agent handoffs, and scale transitions.

The four mechanisms are not independent layers. They form an integrated system. Context integrity (Mechanism 1) ensures that what the Governance Layer intended is what the Agentic Execution Layer acts on. Structured task decomposition (Mechanism 2) ensures that Governance Layer intent is translated into executable work without losing the authority structure in the translation. HITL control points (Mechanism 3) ensure that humans remain in the decision loop at structurally defined intervals, not merely when something appears to be going wrong. Iterative feedback loops (Mechanism 4) ensure that what human review reveals propagates back into the system — closing the loop between judgment and execution. The governance model is only as strong as the weakest mechanism.

11 Antonio Carlos Pacagnella Junior, V. R. da Silva, and P. T. Aquino Junior, “Critical Success Factors for Agile Software Development,” *IEEE Transactions on Engineering Management* 71 (2024): 14807–14823, <https://doi.org/10.1109/TEM.2024.3441829>.

12 Justin H. Kuiper, CISSP, “Edge AI Doctrine: Ten Critical Considerations for Edge AI, With Architectural Ramifications, Consequences, and Governance,” Non Sequitur Publishing, 2026, v1.0-preprint, §5 (The Skipjack Protocol), <https://nonsequitur.tech/white-papers/edge-ai-doctrine/>.

3.2 MECHANISM 1 — CONTEXT INTEGRITY AND EPISTEMIC CONTINUITY

Paper Two of this series established that AI systems are subject to structural epistemic vulnerabilities: semantic compression reduces rich context to impoverished summaries; context saturation creates recency bias when the full information environment exceeds model capacity; confident misalignment produces outputs that look correct while being wrong in subtle ways that are hard for the receiving agent or human to detect.¹³ These vulnerabilities do not disappear in an agentic operating model. They compound: each handoff between agents is an opportunity for semantic compression; each sprint boundary is an opportunity for context loss; each new agent entering a workflow is an opportunity for confident misalignment to replicate.

Mechanism 1 is the Skipjack Protocol's architectural response. Its core principle: context must travel with provenance. When a task moves from the Governance Layer to the Orchestrator, from the Orchestrator to the Agentic Execution Layer, or from one agent to another, the receiving party must receive not just the task content but the chain of intent, constraints, and decisions that produced it.

The implementation structure is the *context integrity manifest*: a structured record that accompanies every task through its lifecycle. The manifest records what the Governance Layer originally intended, what constraints were active at decomposition, what was deliberately excluded from scope, what decisions the Orchestrator made in translating intent into task specification, and what context has been consumed since task creation. The receiving agent does not start from a fresh summary. It inherits a provenance chain.

This is a stronger requirement than context passing. Context passing — forwarding prior conversation content or appending task history — is necessary but not sufficient. Context passing without provenance structure is susceptible to the same compression and summarization errors it is trying to prevent: the summarizer compresses, the compressor loses constraints, and the downstream agent operates on an attenuated version of the original intent. The manifest enforces structure: what the Governance Layer said, not someone's summary of what the Governance Layer said.

Enforcement at the system level: the Orchestrator maintains the context integrity manifest for every active task. No task transfer is complete without manifest acknowledgment from the receiving party. The receiving agent must confirm not just receipt but constraint acknowledgment — it must demonstrate that it has parsed the constraints in the manifest before execution begins. This is analogous to read-receipts with comprehension confirmation; it prevents the failure mode in which a downstream agent proceeds on the assumption that context transfer was complete when it was not.

§4 provides the full treatment of context integrity as an operational system, including cross-sprint continuity and the DDIL-phase case.

3.3 MECHANISM 2 — STRUCTURED TASK DECOMPOSITION AND ROUTING

The Governance Layer sets intent. The Agentic Execution Layer executes. Between those two functions lies the Orchestrator's core operation: translating intent into executable tasks. This translation is the most governance-sensitive step in the cycle. It is where abstract intent — “build a system that validates user identity with a 99.9% success threshold under adversarial

¹³ Justin H. Kuiper, CISSP, “Epistemic Constraints and Semantic Compression in Natural Language Processing: A Theoretical Foundation for the HGC³AE² Framework,” Non Sequitur Publishing, 2026, v1.0-preprint, <https://nonsequitur.tech/white-papers/epistemic-constraints/>.

conditions” — becomes concrete task specifications that agents can act on. And it is where scope creep, authority bleed, and misaligned execution most easily originate.

Mechanism 2 governs this translation through a structural requirement: no task may enter the Agentic Execution Layer without an Orchestrator-generated task specification that contains five elements.

Scope. A precise description of what the task is and is not. The boundaries of the task are as important as its content. An agent that cannot identify where its task ends and the next task begins is an agent that will make scope decisions it is not authorized to make.

Constraints. The Governance Layer parameters that apply to this specific task. Not a general policy document — the specific constraints that are operative here. “Do not call external APIs without pre-approval.” “Output must conform to the data schema in memory node 7.” “Execution must complete within the authorized time window.” Constraints are not suggestions.

Output format. What a complete, correct output looks like for this task. This prevents the failure mode in which an agent produces output that is internally coherent but incompatible with downstream task inputs, creating an integration failure that cascades forward.

Escalation trigger. The specific conditions under which the agent must halt and surface an escalation signal to the Orchestrator rather than continuing to execute. This is the mechanism by which the pre-authorized decision envelope from Paper Four’s governance pillar¹⁴ is implemented in the Skipjack context: the envelope boundary is specified in the task, not inferred by the agent at runtime.

Governance Layer owner. The specific human authority responsible for this task and any escalation that arises from it. Not “the Governance Layer” as an abstraction — the named accountable party. Accountability without attribution is not accountability.

Decomposition that omits any of these five elements is structurally incomplete. An agent executing against an incomplete task specification is not executing within the Skipjack Protocol — it is operating in the gap the protocol was designed to close.

Routing — the assignment of task specifications to appropriate Agentic Execution Layer agents — is the Orchestrator’s second core function. Routing decisions are bounded by the Governance Layer’s intent parameters: the Orchestrator may route tasks to agents it has authority to dispatch, using methodologies within the Governance Layer’s sanctioned approach, in an order consistent with the Governance Layer’s prioritization. Routing is not autonomous scheduling. It is governed dispatch.

3.4 MECHANISM 3 — HUMAN-IN-THE-LOOP CONTROL POINTS

The most common objection to HITL governance in agentic systems is throughput: if humans must review every agent action, the efficiency gain of agentic execution is eliminated. This objection misunderstands what structural HITL requires. The Skipjack Protocol’s HITL control points are not positioned at every execution step. They are positioned at the *governance boundary* — where execution authority crosses a threshold that requires human judgment to validate or redirect.

Within a pre-authorized decision envelope, agents execute without per-step human review. This is not a governance concession — it is the architectural benefit of pre-authorization. The Governance Layer has defined what the agent may do; the agent does it. Human review

¹⁴ Kuiper, “Alistair Prime in a Box,” §2.5 (Governance and Policy Enforcement) and §4.3 (Decision Envelopes and Scope Boundaries).

activates when execution approaches or crosses the envelope boundary, when outputs are ready for integration into persistent memory, and at the structural intervals (sprint review, retrospective) that are HITL by design.

Three HITL control point types are defined in the Skipjack Protocol.

Approval gate. Before a high-stakes task enters the Agentic Execution Layer, the Governance Layer must explicitly approve the task specification. High-stakes is defined by criteria set by the Governance Layer at sprint planning — tasks that produce outputs used as inputs to irreversible operations, tasks that touch external systems outside the established scope, tasks that exceed defined resource thresholds. The approval gate is not a rubber stamp; it is the Governance Layer confirming that the Orchestrator's decomposition of intent into task specification matches what the Governance Layer actually intended. Mismatches at the approval gate are far less costly to correct than mismatches discovered after execution.

Scope checkpoint. During execution of a long-running task, the Orchestrator surfaces a scope question to the Governance Layer when the task's progress indicates it may be approaching an envelope boundary. The agent does not determine whether it has exceeded its scope — that is not within its authority. The agent surfaces a signal; the Orchestrator evaluates whether to escalate; the Governance Layer decides. The agent pauses execution in the interim. Unilateral continuation past a potential scope boundary is a protocol violation.

Completion review. After a task completes, before its output is accepted into persistent memory or used as input to a subsequent task, the Governance Layer reviews execution outputs against the original intent. This is the sprint review ceremony instantiated at the task level for outputs that carry governance significance. Routine task outputs may be reviewed in batches at sprint review rather than individually; the Governance Layer specifies which outputs require individual review at sprint planning.

The structural property of HITL is not about frequency. It is about positioning. HITL that exists at the convenience of execution throughput is not governance. HITL positioned at the governance boundary — where the authority of the Agentic Execution Layer ends and the authority of the Governance Layer must be re-engaged — is the mechanism through which human judgment remains operative in the system even as agentic execution scales.

3.5 MECHANISM 4 — ITERATIVE FEEDBACK LOOPS

The retrospective is the most underappreciated ceremony in Agile Scrum. Sprint planning gets attention because it determines what gets built. Sprint review gets attention because it surfaces what was built versus what was intended. But the retrospective is where the system learns: what the sprint revealed about how the team works, what the review revealed about gaps between the team's process and the product's needs, what corrections are needed before the next sprint begins.

In the Skipjack Protocol, the retrospective's function becomes the feedback loop mechanism. The closing of this loop — from human review outputs back into the system's persistent memory and task routing logic — is the mechanism through which the governed collaboration model improves over time. Without it, the system can deliver governed outputs in each sprint cycle but cannot learn from what those outputs reveal. The persistent memory accumulates content but not understanding. The task routing logic remains static regardless of what patterns emerge. The Governance Layer makes the same types of interventions in Sprint N+1 that it made in Sprint N, because the system has not integrated the correction.

The feedback loop closes in four steps.

Review produces feedback. The sprint review (HITL control point type 3) produces explicit feedback: outputs that satisfied the Governance Layer's intent are accepted; outputs that did not are documented with the specific dimension on which they failed — scope deviation, constraint violation, output format mismatch, or quality threshold failure.

Feedback is categorized. Feedback is categorized by what it implies for the system: memory schema update (new context is now relevant that was not captured), routing logic update (the task was assigned to the wrong type of agent, or the decomposition was under-specified), constraint update (a constraint needs to be added, tightened, or relaxed), or process update (the sprint cycle structure itself needs adjustment).

Governance Layer approves updates. The Orchestrator proposes specific updates to persistent memory and routing logic based on categorized feedback. The Governance Layer reviews and approves. Memory and routing updates are not automatic — they are governed. The Orchestrator does not self-modify; it proposes, and the Governance Layer disposes. This prevents the failure mode in which agentic systems update their own operating parameters based on their own performance evaluations — a mechanism for authority creep that the Skipjack Protocol explicitly rejects.

Updates take effect at the next sprint. Approved updates are applied before the next sprint planning cycle begins. The next sprint's task decomposition and routing operate on the updated parameters. The feedback loop closes.

The iterative property is critical. The Skipjack Protocol is not a fixed-configuration governance model. It improves through governed iteration. The governance model at Sprint N+10 is more finely calibrated than at Sprint N+1 — not because agents have been given more autonomy over time, but because human review has been integrated into the system's operating parameters ten times. The collaboration becomes more effective through iteration, and the iteration is governed throughout.

The distinction between Skipjack's feedback loop and the self-reflection mechanisms in the recent agent literature is significant. Reflexion (Shinn et al. 2023, NeurIPS 2023)¹⁵ introduced verbal reinforcement learning for language agents: after a task attempt, the agent generates natural-language reflection on its own performance and uses that reflection to update its approach to subsequent attempts. Reflexion's self-reflection is agent-internal and unmoderated — the agent is both the source of the performance evaluation and the recipient of the update. There is no external authority that reviews the reflection before the agent acts on it.

The Skipjack feedback loop requires Governance Layer approval before any update to persistent memory or routing logic takes effect. The distinction is structural: in Reflexion, the agent updates its own state based on its own self-assessment. In Skipjack, the agent proposes; the Governance Layer disposes. This is not a performance tradeoff — agent-internal self-reflection may produce faster iteration — it is a governance commitment. A system in which agents update their own operating parameters based on their own performance evaluations is a system in which agent behavior can drift from Governance Layer intent without any human observation of the drift. Skipjack's feedback loop requires a human to see the proposed update and approve it before the system acts on it. The human remains in the loop not as a speed bump but as the authority that owns the decision.

¹⁵ Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *Advances in Neural Information Processing Systems* 36 (NeurIPS 2023), https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

3.6 THE MECHANISMS AS INTEGRATED SYSTEM

The four mechanisms address distinct failure modes but operate as a unit. Context integrity without structured decomposition fails at the translation step: the manifest documents the intent, but the task specification doesn't encode the constraints, so downstream agents lose governance fidelity. Structured decomposition without HITL control points fails at the review boundary: tasks are well-specified, but there is no structural moment at which human judgment validates what was produced. HITL control points without feedback loops fail at the learning boundary: human review happens, generates corrections, but those corrections don't persist across sprint cycles. Feedback loops without context integrity fail at the next cycle's start: the updated persistent memory travels forward, but the provenance that makes it interpretable does not.

Together, the four mechanisms close the governance loop. Context integrity ensures that intent survives translation. Structured decomposition ensures that intent is translated correctly. HITL control points ensure that human judgment re-engages at the governance boundary. Feedback loops ensure that what human judgment reveals is integrated back into the system. The result is a governed collaboration model that maintains human authority not through continuous intervention but through structural positioning of human judgment at the moments where its authority is most needed.

4. CONTEXT INTEGRITY AND EPISTEMIC CONTINUITY

4.1 WHY CONTEXT INTEGRITY IS A PROTOCOL CONCERN, NOT A TOOLING CONCERN

The immediate response to context loss in AI systems is to add more context: extend the context window, use retrieval-augmented generation to surface relevant prior state, implement summarization pipelines that preserve more information than naive truncation. These are legitimate engineering improvements. They are not solutions to the governance problem that context integrity is designed to address.

The governance problem is this: when an agent acts, it acts on the context it has. If that context is incomplete, compressed, or attenuated, the agent's action reflects the incomplete version of what the Governance Layer intended, not the complete version. The agent has no way to know which parts of the original intent are missing — it can only act on what it received. This is not a failure of the agent. It is a structural consequence of the information environment in which it operates. Engineering improvements that extend context window or improve retrieval can reduce the probability of context loss; they cannot eliminate the structural risk, and they cannot make the loss detectable after the fact without a provenance system.

Context integrity is the protocol-level response to this structural risk. It does not replace engineering improvements; it builds on them. The context window can be as large as the system can afford; retrieval can be as sophisticated as the system can implement. Context integrity adds the requirement that the chain of intent and constraints that produced each task is structurally preserved and structurally verified at every handoff — so that when context loss does occur, it is detectable, attributable, and correctable rather than silent.

The key distinction is between best-effort context passing and protocol-enforced provenance. Best-effort context passing asks: how much context can we forward? Protocol-enforced provenance asks: how do we know the context that was forwarded is the context that

was intended, and how do we detect when it is not? The Skipjack Protocol answers the second question. The manifest structure implements the answer.

4.2 THE CONTEXT INTEGRITY MANIFEST IN PRACTICE

Provenance tracking at the state layer is established practice. The W3C PROV Data Model (2013)¹⁶ and PROV Ontology (2013)¹⁷ define canonical provenance vocabulary: entity, activity, agent, and the wasGeneratedBy, used, and wasAttributedTo relations that trace how information artifacts were produced. ProvONE (DataONE, 2015)¹⁸ extends this model to scientific workflow provenance. CWLProv (Khan et al. 2019)¹⁹ implements workflow provenance in production using W3C PROV vocabulary, demonstrating that provenance capture in production execution pipelines is feasible and practically useful. The biomedical provenance scoping review (Gierend et al. 2024)²⁰ confirms that provenance infrastructure for complex, multi-step data workflows is a mature research area.

LangGraph's checkpointing system captures state provenance at the harness layer: thread IDs, state deltas, and checkpoint timestamps provide a traceable record of state evolution across a workflow. This is operationally analogous to workflow provenance in the data pipeline tradition.

The Skipjack context integrity manifest is not a novel invention of provenance. It is an elevation of provenance to a governance artifact with mandatory fields that existing provenance infrastructure does not enforce. The four fields of the manifest — original Governance Layer intent, explicit exclusion record, constraint registry in original form, handoff receipt log with comprehension acknowledgment — are not present in W3C PROV vocabulary, LangGraph checkpointing, or CWLProv workflow records, because those systems are designed to trace what happened, not to enforce that original human intent was preserved through what happened. The manifest adds to the provenance tradition the governance obligation: not just “here is what occurred” but “here is the original intent against which what occurred must be evaluated.” That obligation is what makes the manifest a governance artifact.

The context integrity manifest is a structured record that accompanies every task through its lifecycle. Its structure is designed around the failure modes it prevents.

16 World Wide Web Consortium, *PROV-DM: The PROV Data Model*, W3C Recommendation, April 30, 2013, <https://www.w3.org/TR/prov-dm/>.

17 World Wide Web Consortium, *PROV-O: The PROV Ontology*, W3C Recommendation, April 30, 2013, <https://www.w3.org/TR/prov-o/>.

18 DataONE, *ProvONE: DataONE PROV Extension for Scientific Workflow Provenance*, specification, 2015, <http://purl.dataone.org/provone/2015/01/15/ontology>. Cited as a domain-specific provenance specification, not a peer-reviewed publication.

19 Farah Zaib Khan, Stian Soiland-Reyes, Richard O. Sinnott, Andrew Lonie, Carole Goble, and Michael R. Crusoe, “Sharing interoperable workflow provenance: A review of best practices and their practical application in CWLProv,” *GigaScience* 8, no. 11 (2019): giz095, <https://doi.org/10.1093/gigascience/giz095>.

20 Kerstin Gierend, Frank Krüger, Sascha Genehr, Francisca Hartmann, Fabian Siegel, Dagmar Waltemath, Thomas Ganslandt, and Atinkut Alamirrew Zeleke, “Provenance Information for Biomedical Data and Workflows: Scoping Review,” *Journal of Medical Internet Research* 26 (2024): e51297, <https://doi.org/10.2196/51297>.

The manifest contains a *provenance chain*: each step in the task's history, from Governance Layer intent through Orchestrator decomposition to Agentic Execution Layer assignment, documented with the specific decisions made at each step and the reasoning that informed them. The chain is append-only — no step in the chain is modified by subsequent steps. When the task completes and enters persistent memory, the full chain is preserved alongside the output. A Governance Layer reviewer reading the output can trace every decision that produced it.

The manifest contains an *explicit exclusion record*: what the Orchestrator deliberately chose not to include in the task specification, and why. This addresses a specific failure mode: the downstream agent cannot know what was excluded if the exclusion is not documented. An agent that receives a task specification covering dimensions A, B, and C does not know whether dimension D was considered and excluded or whether it was never considered. The exclusion record answers this question. When an agent encounters dimension D during execution and it is not in the task specification, the exclusion record tells the agent whether to proceed (D was excluded by design), to surface a scope question (D may have been overlooked), or to escalate (D is in the scope checkpoint trigger conditions).

The manifest contains a *constraint registry*: the specific Governance Layer parameters operative for this task, in their exact original form. Not paraphrased, not summarized, not extracted from a broader policy document — the specific constraints as the Governance Layer expressed them. When the agent consults the constraint registry during execution, it is reading the Governance Layer's actual intent, not an interpretation of it.

The manifest contains a *handoff receipt log*: a record of every agent that has received this task, the context manifest version they received, and their explicit acknowledgment of constraint receipt. If an agent proceeds with execution and it is later determined that the constraints were not operative during that execution, the handoff receipt log identifies exactly where the failure occurred: was the manifest not transmitted, was it transmitted in an attenuated version, was it received but not acknowledged, or was it acknowledged but not acted on? Each failure mode has a different corrective response, and the handoff receipt log makes them distinguishable.

4.3 EPISTEMIC CONTINUITY ACROSS SPRINT BOUNDARIES

Within a sprint, the context integrity manifest travels with each task. Between sprints, the epistemic continuity problem has a different character: the question is not how context travels with a specific task but how the accumulated context from Sprint N informs Sprint N+1's task decomposition.

The persistent memory system is the mechanism, but persistent memory without governance is not epistemic continuity — it is epistemic accumulation. The difference is curation. An uncured persistent memory grows in volume while declining in navigability: more content, but less clarity about which content is operative, which is superseded, and which is relevant to the upcoming sprint's objectives. Epistemic accumulation without curation produces the same degradation it was meant to prevent, just more slowly.

Epistemic continuity requires memory curation as a Governance Layer function. At the end of each sprint, before the next sprint planning cycle begins, the Governance Layer reviews the outputs of the sprint review and retrospective and makes explicit decisions about the persistent memory schema: what new information is added, what prior information is superseded or archived, what context is marked as operative for the upcoming sprint's task decomposition. The Orchestrator implements these decisions; it does not make them.

This curation function is not bureaucratic overhead. It is the mechanism through which the Governance Layer exercises authority over what the system knows and how it uses that knowledge. A Governance Layer that delegates memory curation to the Orchestrator or the Agentic Execution Layer has delegated authority over the system's epistemic state. The system's epistemic state determines what tasks are proposed, how they are decomposed, and what constraints are applied. Authority over epistemics is authority over execution. It belongs at the Governance Layer.

4.4 CROSS-AGENT CONTEXT FIDELITY

In multi-agent systems, tasks are frequently handed from one agent to another: a research agent surfaces findings that a synthesis agent integrates, a synthesis agent produces a draft that an evaluation agent reviews, an evaluation agent produces feedback that a revision agent incorporates. Each handoff is an opportunity for the epistemic integrity of the original Governance Layer intent to degrade.

The failure mode is subtle. Each individual agent in the chain may operate competently on the context it received. The research agent may produce excellent findings. The synthesis agent may produce an excellent synthesis of those findings. The evaluation agent may produce an accurate evaluation of the synthesis. At no step does any agent make an error by its own assessment criteria — yet the revision agent's output may be systematically misaligned with the original Governance Layer intent, because each handoff introduced a small compression or reinterpretation that compounded across the chain.

Cross-agent context fidelity addresses this through two mechanisms. First, the manifest protocol applies at every handoff, not just at the first Orchestrator-to-agent transfer. Each agent in the chain is responsible for transmitting the full manifest — including its own additions to the provenance chain — to the next agent, and the next agent is responsible for manifest acknowledgment before executing. The manifest is not static; it grows as the task moves through the chain, accumulating the decisions and context additions each agent contributed.

Second, the Orchestrator monitors the manifest chain for cumulative compression: situations in which each individual handoff is within acceptable parameters but the aggregate drift from the original intent exceeds the governance threshold. If cumulative compression is detected, the Orchestrator does not attempt to correct it by reinserting context at the next handoff — it escalates to the Governance Layer, which re-grounds the task chain from the original intent. The recovery mechanism for cumulative compression is re-derivation, not incremental correction. Incremental correction on a compressed baseline produces a corrected version of the compressed content, not a recovery of the original.

4.5 CONTEXT INTEGRITY UNDER DDIL CONDITIONS

Paper Four established that self-sufficient cognitive nodes must maintain operational capability under denied, degraded, intermittent, and limited connectivity conditions. Context integrity under DDIL introduces a specific challenge: when a node is operating in disconnected mode, the Orchestrator may not be reachable to maintain the manifest chain for a task that spans a connectivity gap.

The Skipjack Protocol handles this through the pre-authorized execution envelope — the same mechanism Paper Four's governance pillar uses to maintain governance in the absence of a live human connection.²¹ Before DDIL conditions begin, the Orchestrator generates a local

²¹ Kuiper, "Alistair Prime in a Box," §4 (Human Governance in Disconnected Operations), particularly §4.3 (Decision Envelopes) and §4.4 (Asynchronous Oversight Mechanisms).

context integrity manifest for all tasks authorized to execute during the disconnected period. The manifest is locally stored on the node and travels with all task executions during the DDIL phase. The pre-authorized manifest functions as a bounded permission set: the node may execute tasks that are specified in the manifest with the constraints and output formats it specifies. It may not execute tasks outside the manifest scope.

When connectivity is restored, the DDIL-phase manifest — including the full provenance chain of all executions that occurred — is transmitted to the Orchestrator for integration into the central system. The Governance Layer reviews DDIL-phase outputs against the pre-authorized manifest at the next completion review. Outputs within mandate are accepted; outputs that deviated from the manifest trigger an audit of what caused the deviation. The reconnection event is itself a HITL control point.

This is the protocol-level expression of what Paper Four described architecturally: governance is pre-positioned, not continuous. The manifest structure makes the pre-positioning explicit and verifiable.

4.6 WHAT CONTEXT INTEGRITY IS NOT

To complete the treatment: context integrity is not a surveillance mechanism, and it should not be implemented as one.

The manifest is a provenance record, not an activity log. Its purpose is to preserve the chain of intent, constraints, and decisions that produced each task — not to record every action an agent took during execution in exhaustive detail. Exhaustive execution logging has legitimate uses in debugging and audit systems; it is not what context integrity requires. An overly granular manifest imposes implementation overhead that reduces adoption and creates navigability problems that undermine the provenance clarity the manifest is supposed to provide. The manifest is designed to be the minimum structured record that allows a Governance Layer reviewer to reconstruct whether an output reflects the original intent. That is its standard.

Context integrity is not a substitute for agent capability. A well-maintained manifest allows a Governance Layer reviewer to detect when an agent's output deviated from intent. It does not improve the quality of an agent's execution. Mechanism 1 addresses the epistemics of context transfer; it does not address the capability of the agents doing the executing. The Skipjack Protocol is a governance framework, not a capability framework. Governance without sufficient capability produces well-documented failures. Both are required.

Context integrity is not a lock against the Governance Layer's own errors. If the Governance Layer sets intent that is internally inconsistent, the manifest faithfully preserves and transmits that inconsistency throughout the task chain. The manifest is only as useful as the intent it documents. This is not a limitation of the mechanism — it is a statement about the irreducibility of human judgment in the system. The mechanism cannot substitute for judgment; it can only ensure that judgment, once applied, is preserved.

5. HITL CONTROL POINTS AND ITERATIVE FEEDBACK LOOPS

5.1 WHY HITL MUST BE STRUCTURAL

Human-in-the-loop provisions that activate only at the operator's discretion are not governance — they are post-hoc review dressed as oversight. The distinction matters operationally. A system in which human review is technically possible but structurally optional

will, under schedule pressure, resource constraint, or increasing agent reliability, drift toward less human review over time. The governance model degrades not through any single decision but through the cumulative effect of individual decisions to skip the review. By the time the degradation is visible, the system has been operating outside governance for some period that the audit log cannot fully account for.

The Skipjack Protocol defines HITL as a structural property: execution cannot proceed past defined control points without human review. This is not a policy preference. It is an architectural commitment. Control points are built into the task lifecycle as checkpoints the Orchestrator cannot bypass — not because the Orchestrator is untrustworthy, but because the governance model does not assign the Orchestrator authority to make that determination. The human at the Governance Layer has that authority. The system is designed to require its exercise.

The growing peer-reviewed literature on human-in-the-loop AI systems provides a structural frame for this commitment. The 2026 systematic review of HITL concepts, methods, and applications published in *Entropy*²² identifies the organizational and technical conditions under which HITL mechanisms succeed and fail. The failure modes are consistent: HITL mechanisms that do not carry structural enforcement — that rely on workflow discipline rather than system architecture — erode at the seams of organizational pressure. The Skipjack Protocol responds to this finding by making HITL architectural: the control point is not a step the workflow recommends; it is a gate the system enforces.

5.2 CONTROL POINT TAXONOMY

The Skipjack Protocol defines three types of HITL control point, distinguished by when in the task lifecycle they activate and what human authority they invoke.

Approval gates occur before execution begins. The Governance Layer reviews the Orchestrator's task decomposition plan — the specification of what the Agentic Execution Layer will do, in what sequence, within what constraints — and approves or revises it before any execution step commences. The approval gate is the primary mechanism by which the Governance Layer's intent is translated from high-level objective into an executable specification the Agentic Execution Layer can act on. An approval gate that is rushed, poorly documented, or approved without genuine review of the decomposition plan is a governance failure at the source. The quality of what the system produces is bounded by the quality of the Governance Layer's engagement at the approval gate.

Scope checkpoints occur during execution when the Orchestrator identifies that a task is approaching the boundary of its pre-authorized scope. The scope checkpoint is not a failure signal — it is the governance mechanism functioning correctly. The Orchestrator surfaces the scope question to the Governance Layer; the Governance Layer decides whether to extend the scope, revise the task specification, or halt and re-plan. An Orchestrator that routes past the scope boundary without surfacing a checkpoint has exceeded its authority. An Orchestrator that surfaces checkpoints routinely and without judgment has abdicated its responsibility to manage execution within approved parameters. The correct behavior is discriminating escalation: surface the scope question when the boundary is genuinely uncertain, proceed when the question is unambiguous.

Completion reviews occur after execution concludes, before outputs are accepted into persistent memory or used as inputs to subsequent tasks. The completion review is the gate through which agentic outputs become part of the organizational knowledge base. This is where

22 "Human-in-the-Loop Artificial Intelligence: A Systematic Review of Concepts, Methods, and Applications," *Entropy* 28, no. 4 (2026): 377, <https://doi.org/10.3390/e28040377>.

the Governance Layer exercises authority over epistemic accretion: what the system has learned from this execution cycle, what it will remember, and what will inform the next cycle's task planning. A completion review that approves all outputs mechanically is not a governance mechanism — it is throughput processing with a human signature. The review requires genuine evaluation against the intent stated at the approval gate.

5.3 ESCALATION PATHS

When the Agentic Execution Layer encounters a decision that exceeds its task specification, the escalation path is defined: halt and surface to the Orchestrator. The Orchestrator determines whether the issue falls within its authority to resolve (a task re-routing decision, a resource allocation choice within approved parameters) or whether it requires Governance Layer involvement. If Governance Layer involvement is required, the Orchestrator surfaces the issue with a structured escalation: what the agent encountered, what options are available, what the Orchestrator recommends, and what the decision requires.

This path makes escalation a routine mechanism, not a crisis signal. In systems where escalation is treated as a sign of agent failure, agents are structurally incentivized to resolve scope questions without surfacing them — to make the decision and document it later, or to stretch the task specification until the decision fits within it. The Skipjack Protocol explicitly rejects this incentive. Escalation is the governance mechanism working. Failure is an agent making a scope decision it is not authorized to make.

The Governance Layer's response to escalation should be prompt and decisive. A governance model that escalates correctly but reviews slowly creates a different failure mode: agents in a hold state while awaiting human review, backlogs of escalated decisions, and pressure to reduce escalation thresholds to maintain throughput. The solution is not to reduce HITL — it is to invest in Governance Layer capacity proportional to the escalation volume the system generates.

5.4 FEEDBACK LOOP MECHANICS

The feedback loop closes the governance cycle from sprint review back to sprint planning. Its mechanics are specified in §3.5, where the four-step structure — review produces feedback, feedback is categorized, Governance Layer approves updates, updates take effect at next sprint — is defined. The focus here is on what makes the loop function as a learning mechanism rather than a compliance ritual.

Feedback categorization determines whether the loop is generative or merely reactive. Feedback that identifies only output failures — this deliverable did not meet the specification — produces corrections. Feedback that identifies structural patterns — the Agentic Execution Layer consistently underperforms when tasks involve domain X, the Orchestrator's decomposition systematically under-specifies constraint Y — produces system improvement. The Governance Layer's role in the feedback loop is not to issue corrections; it is to identify what the pattern of corrections reveals about the system's operating model and to approve the updates that address root causes rather than surface symptoms.

The Governance Layer's approval authority over memory and routing updates (§3.5) is the mechanism that prevents feedback loops from enabling authority creep. In systems where agents can update their own operating parameters based on performance feedback, the feedback loop becomes a mechanism through which agents expand their own decision authority — a process that can be individually justified at each step while collectively drifting away from the Governance Layer's original intent. Skipjack requires Governance Layer approval for any

update to persistent memory or routing logic that results from the feedback cycle. The loop improves the system; the Governance Layer controls the improvement.

5.5 SCALING HITL WITHOUT BOTTLENECKING

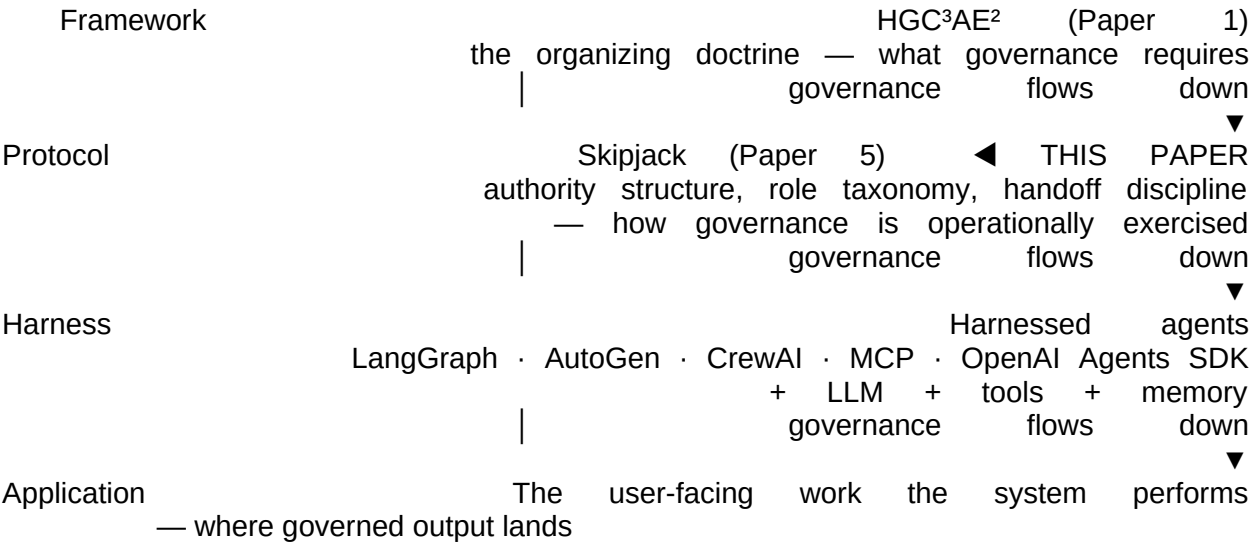
The objection to HITL at scale is well-formed: if every task requires human review at three control points, the human review capacity becomes the binding constraint on system throughput, eliminating the productivity benefit of agentic execution. The Skipjack Protocol's response is architectural.

HITL is applied at the governance boundary, not at every execution step. The approval gate authorizes a task decomposition plan. Within that plan, individual execution steps — tool invocations, memory reads, sub-task completions — proceed without requiring per-step human review, provided they operate within the approved specification. HITL activates when execution approaches the scope boundary, not throughout execution. This is the same structural principle as the pre-authorized decision envelope in Paper Four: governance is pre-positioned at the boundary, not applied uniformly across the interior.

Approval gates scale with task portfolio design. A governance model that requires the Governance Layer to review every task individually at approval will bottleneck at portfolio scale. The solution is structured pre-authorization: the Governance Layer defines categories of approved task specifications — templates with pre-approved parameters for recurring task types — which the Orchestrator can invoke without individual approval for each instance. Novel tasks, high-stakes tasks, and tasks approaching constraint boundaries require individual approval. Routine tasks within established parameters proceed on category approval. The Governance Layer's review load scales with novelty and risk, not with volume.

6. SCALING ACROSS THE HARNESS STACK

Skipjack is the Protocol layer of a four-tier architecture that describes how AI governance flows from doctrine into production deployment:



The protocol descends from the Framework layer (HGC³AE², Paper 1) and governs the Harness layer below it. Skipjack does not compete with LangGraph, AutoGen, or CrewAI — it specifies the governance authority structure within which any of those substrates must operate.

A LangGraph-based system can be made Skipjack-compliant; this paper defines what compliance requires.

6.1 THE GOVERNANCE GAP IN THE CURRENT HARNESS LANDSCAPE

The agent harness frameworks listed above represent the current state of production AI deployment. They are capable, widely adopted, and improving rapidly. They share a structural characteristic: governance is not their primary concern.

The four most-deployed production harness frameworks — LangChain/LangGraph, CrewAI, the Model Context Protocol, and the OpenAI Agents SDK — have no peer-reviewed publication.²³ Governance design, safety architecture, and accountability structure receive minimal treatment in their documentation and none in the academic literature, because there is no academic literature on them to speak of.

The peer-reviewed agent literature is concentrated in research-oriented multi-agent papers: AutoGen (Wu et al. 2024),²⁴ MetaGPT (Hong et al. 2024),²⁵ ChatDev (Qian et al. 2024),²⁶ ReAct (Yao et al. 2023),²⁷ and Reflexion (Shinn et al. 2023).²⁸ These papers address capability — how agents reason, coordinate, and improve through reflection. They do not address governance —

23 As of the writing of this paper, the four most-deployed production agent harness frameworks — LangChain/LangGraph (Harrison Chase et al., LangChain AI), CrewAI (João Moura et al.), the Model Context Protocol (Anthropic), and the OpenAI Agents SDK (OpenAI) — have no peer-reviewed publication. This observation should be updated if peer-reviewed publications appear for these frameworks.

24 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang, “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations,” in *Proceedings of the First Conference on Language Modeling (COLM 2024)*, <https://openreview.net/forum?id=BAakY1hNKS>.

25 Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber, “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024, Oral)*, <https://openreview.net/forum?id=VtmBAGCN7o>. Oral acceptance rate ~1.2% (86 orals / 7,304 submissions).

26 Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun, “ChatDev: Communicative Agents for Software Development,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024) — Long Papers*, 15174–15186, <https://aclanthology.org/2024.acl-long.810>.

27 Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proceedings of the Eleventh International Conference on Learning Representations (ICLR 2023)*, https://openreview.net/forum?id=WE_vluYUL-X.

28 Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.

who holds authority over what decisions, how that authority is maintained as the system scales, or what a human must be able to verify about what an agent did and why.

This asymmetry is not a criticism of the harness frameworks. It is a structural observation about where the research investment has gone. The Skipjack Protocol fills the governance gap that the harness literature has not addressed. This paper is the fifth in that series, addressing the governed operating model across agent deployments.

The multi-agent foundations on which the harness layer is built have deeper roots in the Multi-Agent Systems literature than the current LLM-agent discourse acknowledges. Wooldridge's *An Introduction to MultiAgent Systems* (2009)²⁹ establishes the agent, role, and interaction taxonomy that the LLM-agent literature has largely reinvented without citation. The Skipjack Protocol connects to this literature deliberately: the role taxonomy of §2 is a refinement of MAS role theory, grounded in Scrum's authority structure (§2.7), applied to the governance of LLM-based agentic systems. The Skipjack role remap is not invented from scratch; it extends a mature tradition.

6.2 ROLE-BY-ROLE HARNESS POSITIONING

The Skipjack Protocol's four-tier role taxonomy has a direct analogue in the agent harness layer. The table below maps each Skipjack role to its harness-layer analogue and identifies what Skipjack adds:

Skipjack Role

Harness Analogue

What Skipjack Adds

Governance Layer

AutoGen UserProxyAgent; CrewAI Manager

Named accountable humans with documented authority; cannot be substituted by a proxy agent standing in for a human

Orchestrator

²⁹ Michael Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. (Chichester: Wiley, 2009), ISBN 9780470519462.

LangGraph supervisor node; CrewAI process manager; MetaGPT SOP orchestrator (Hong 2024)³⁰

Bounded by Governance Layer parameters; does not self-direct or extend its own mandate

Agentic Execution Layer

CrewAI agents; AutoGen AssistantAgent; ChatDev agent roles (Qian 2024)³¹

Execution within governed scope; mandatory escalation trigger per task specification

Persistent memory + task queues

LangGraph state checkpointing; AutoGen conversation queues

Governance Layer authority over what persists across sprint boundaries; agent does not self-promote

Each row represents a governance contract, not a technical specification. The harness analogue provides the mechanism; the Skipjack addition defines what the mechanism must enforce for the implementation to be Governance Layer-compliant.

Governance Layer. AutoGen’s UserProxyAgent and CrewAI’s Manager role occupy the position in their respective frameworks where human intent enters the system. They are technically comparable to the Skipjack Governance Layer in that they represent a designated entry point for human direction. The Skipjack addition is specificity about what “human” means: a named, accountable person with documented authority over the specific mission domain, whose instructions are traceable through the context integrity manifest (§4). A UserProxyAgent configured to run without human intervention — a common deployment pattern for automation workflows — is not a Governance Layer implementation. The Skipjack Governance Layer cannot be delegated to an automated agent standing in for a human, regardless of how that agent is labeled.

Orchestrator. LangGraph’s supervisor node pattern and CrewAI’s process manager implement multi-step workflow coordination with role-based task delegation — directly analogous to the Orchestrator’s function. MetaGPT’s Standard Operating Procedure orchestration (Hong et al. 2024)³² adds structured workflow specifications to this pattern, moving

30 Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber, “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024, Oral)*, <https://openreview.net/forum?id=VtmBAGCN7o>. Oral acceptance rate ~1.2% (86 orals / 7,304 submissions).

31 Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun, “ChatDev: Communicative Agents for Software Development,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024) — Long Papers*, 15174–15186, <https://aclanthology.org/2024.acl-long.810>.

32 Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber, “MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework,” in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024, Oral)*, <https://openreview.net/forum?id=VtmBAGCN7o>. Oral acceptance rate ~1.2% (86 orals / 7,304 submissions).

toward the kind of decomposition discipline the Skipjack Protocol requires. The Skipjack addition is the constraint on self-direction: the Orchestrator executes within Governance Layer parameters and surfaces scope questions that exceed those parameters. MetaGPT's SOP-based orchestrator constrains agents through predefined workflows; Skipjack constrains the Orchestrator itself through Governance Layer authority. The constraint chain runs from humans to Orchestrator to agents, not from SOPs to agents.

Agentic Execution Layer. CrewAI's agent roles, AutoGen's AssistantAgent pattern, and ChatDev's role-based agent composition (Qian et al. 2024)³³ all implement multi-agent task execution with defined agent specializations. These frameworks are technically well-suited to the Agentic Execution Layer function. The Skipjack addition is the escalation contract: each agent in the Agentic Execution Layer has a defined scope boundary, a defined escalation trigger, and a defined escalation path. ChatDev's role specialization — product manager, architect, coder, reviewer — is a strong analogue for the decomposition-by-specialization model; Skipjack adds the governance dimension of what happens when any of those roles encounters a decision outside its scope.

Persistent Memory and Task Queues. LangGraph's state checkpointing and AutoGen's conversation queue management handle the memory and continuity requirements that map to Skipjack's persistent memory and task queue tier. The Skipjack addition is authority over promotion: what enters the persistent knowledge base across sprint boundaries is a Governance Layer decision, not an automatic consequence of execution. Harness frameworks that auto-promote — retaining state that proves useful, expanding the context that persists across sessions — create an epistemic accumulation the Governance Layer no longer fully controls. Skipjack requires that what persists between sprints is what the Governance Layer authorized to persist.

6.3 MULTI-AGENT COORDINATION AND MAS GROUNDING

AutoGen (Wu et al. 2024)³⁴ and MetaGPT (Hong et al. 2024)³⁵ are the best peer-reviewed multi-agent comparators for the Skipjack Protocol. AutoGen's conversational multi-agent model — agents that collaborate through structured dialogue with defined roles and handoff conventions — is structurally analogous to the Skipjack inter-agent communication model. MetaGPT's SOP-based coordination adds workflow structure to the multi-agent interaction

33 Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun, "ChatDev: Communicative Agents for Software Development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024) — Long Papers*, 15174–15186, <https://aclanthology.org/2024.acl-long.810>.

34 Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations," in *Proceedings of the First Conference on Language Modeling (COLM 2024)*, <https://openreview.net/forum?id=BAakY1hNKS>.

35 Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber, "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework," in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR 2024, Oral)*, <https://openreview.net/forum?id=VtmBAGCN7o>. Oral acceptance rate ~1.2% (86 orals / 7,304 submissions).

pattern. Both are production-viable frameworks and represent the peer-reviewed state of the art in multi-agent capability.

The Skipjack Protocol's relationship to these frameworks is not that of an alternative architecture — it is that of a governance layer above them. A Skipjack-compliant implementation running on AutoGen would use AutoGen's multi-agent conversation infrastructure to implement the Orchestrator-to-Execution-Layer communication channel, while Skipjack defines what that channel must carry (task specifications with governance manifests), what it must enforce (scope boundaries with escalation triggers), and what it must produce (structured audit records). Skipjack specifies the governance contract; AutoGen implements the communication substrate. Inter-agent communication protocols below the governance layer are substrate-level concerns outside Skipjack's scope.

6.4 WHAT SKIPJACK REQUIRES OF THE HARNESS SUBSTRATE

A system implementing the Skipjack Protocol on top of a standard agent harness would require the following from the substrate:

Governance Layer representation as a first-class role that cannot be configured to run without human participation — no automated proxies, no unsupervised delegation.

Orchestrator scope enforcement at the task-routing layer: the planner checks whether a proposed task falls within the current Governance Layer authorization before routing it, not after.

Escalation propagation: the harness must support a structured escalation path from Execution Layer agent to Orchestrator to Governance Layer, with the escalation carrying sufficient context for the Governance Layer to make an informed decision without re-examining the full task history.

Governed memory promotion: state persistence across sprint boundaries is gated on a signal from the Governance Layer tier, not determined by the harness's internal relevance or recency scoring.

Context manifest transport: the context integrity manifest (§4) travels with the task specification through every handoff in the execution chain; no agent can receive a task without receiving its manifest.

These requirements do not mandate a specific harness implementation. They are governance contracts that any compliant implementation must honor, regardless of whether the underlying orchestration runs on LangGraph, AutoGen, or a bespoke system. The governance doctrine is substrate-independent; the compliance obligation is not.

7. FUTURE STUDY: TEN RESEARCH VECTORS

The Skipjack Protocol as defined in this paper is a governance doctrine, not a validated implementation. The ten research directions below identify where the doctrine requires empirical grounding, where its claims need formal verification, and where the operative context will likely force extension.

1. Formal specification of HITL control point topologies. The three control point types defined in §5.2 — approval gate, scope checkpoint, completion review — are specified informally in natural language. Formal specification languages for HITL topologies would allow machine-verifiable governance contracts: systems that can check whether their control point configuration satisfies a Skipjack-compliance specification before deployment. Type-theoretic

approaches, process calculi, and formal workflow specification languages are candidate formalisms. The challenge is expressiveness: the governance conditions that define control point topology — when does a task require an approval gate? what constitutes a scope boundary? — are rich enough that any formal encoding will involve tradeoffs between precision and practicality.

2. Empirical measurement of governed collaboration efficiency. The claim that governed collaboration produces better outcomes than ungoverned autonomous execution is foundational to the Skipjack Protocol's value proposition. It has not been empirically demonstrated at scale. The research question is specific: given a representative task portfolio, a defined team composition (Governance Layer human + Orchestrator + Agentic Execution Layer), and a defined governance configuration, does Skipjack-governed execution outperform ungoverned autonomous execution on task quality, failure rate, and recovery cost — and by how much? The answer will vary by task type, team composition, and domain. The Papers 9–10 capstone results paper (the Robo Stack implementation study) is the natural vehicle for this measurement. The research design challenge is establishing the ungoverned baseline without creating an operational environment where ungoverned AI execution is acceptable.

3. Skipjack adaptation for regulated industries. Defense, healthcare, and financial services impose governance requirements on AI systems that partially overlap with Skipjack's structure (audit trails, human approval authorities, scope restrictions) and partially diverge (specific regulatory frameworks, sector-specific authority structures, legally defined accountability chains). Mapping the Skipjack Protocol to DoD AI ethics principles, HIPAA requirements for AI in clinical decision support, and SOX/FINRA constraints on AI in financial advisory functions would produce sector-specific compliance profiles. The research question is whether Skipjack's governance contracts are general enough to serve as a substrate for sector-specific compliance, or whether sector requirements force structural changes to the protocol itself.

4. Adversarial testing of context integrity mechanisms. The context integrity manifest (§4) is designed to preserve the Governance Layer's intent through the task execution chain. A sufficiently capable adversary targeting this mechanism would attempt to introduce drift between the Governance Layer's expressed intent and what the Agentic Execution Layer actually acts on — corrupting the manifest in transit, injecting false exclusion records, or attenuating the constraint registry. Red-team testing of context integrity mechanisms — deliberate attempts to degrade the fidelity of intent transmission through the chain — would identify failure modes that theoretical design misses. This research requires collaboration between AI governance researchers and adversarial machine learning practitioners.

5. Cross-organizational Skipjack interoperability. When two organizations' Skipjack-governed systems must collaborate on a shared task, no single Governance Layer has authority over the full workflow. Authority negotiation between two Governance Layers — establishing which authority structure governs which portion of the shared task, how context manifest handoffs are handled at organizational boundaries, and what audit obligations each party carries — is an open governance design problem. Distributed authorization protocols and inter-organizational trust frameworks from the security literature provide starting points, but the Skipjack-specific requirement that the context integrity manifest travel with the task across organizational boundaries creates novel challenges for both technical implementation and organizational policy.

6. Governance Layer tooling design. The Governance Layer human is a bottleneck if the tooling available to them does not scale with the decision volume the system generates. The research question is: what software interfaces support effective Governance Layer decision-

making without either overwhelming the human with detail or abstracting so much that meaningful oversight is lost? Dashboard design for multi-agent oversight, escalation queue management, context manifest review interfaces, and feedback categorization tools are all open design problems. The governance model's effectiveness is bounded by the cognitive adequacy of the tooling available to the human at the governance boundary.

7. Orchestrator intelligence requirements. The Orchestrator's function — task decomposition, routing, scope monitoring, escalation judgment — requires both domain knowledge and meta-cognitive judgment about when a task is approaching a governance boundary. The research question is how much of this intelligence can be provided by a general-purpose language model versus how much requires domain-specific training or fine-tuning. An Orchestrator that consistently fails to identify scope boundary conditions will either escalate too rarely (governance erosion) or too frequently (throughput collapse). The Orchestrator's calibration to a specific domain's scope boundary topology is likely a deployment-time requirement, not a general-purpose capability.

8. Memory governance audit tooling. As a Skipjack-governed system accumulates episodic and semantic memory across sprint cycles, the Governance Layer's responsibility for curating what persists becomes cognitively demanding. Automated tools that assist with memory curation — flagging candidate entries for review, identifying redundancy, highlighting entries that may conflict with current Governance Layer constraints, and summarizing the epistemic evolution of the knowledge base over time — would reduce the cognitive load without removing human authority. The design constraint is that memory governance tooling must not itself become a source of unverified epistemic drift: tools that recommend memory decisions must be transparent about what criteria they are using and must not pre-decide for the Governance Layer.

9. Human-to-agent ratio calibration for task types. The Skipjack Protocol does not prescribe a specific ratio of human Governance Layer reviewers to agents in the Agentic Execution Layer. The optimal ratio varies by task risk profile, domain complexity, governance checkpoint frequency, and the cognitive demands of Governance Layer review for the task type in question. Empirical research mapping task characteristics to governance configuration requirements — how many Governance Layer humans are needed to review how many agents executing what class of tasks — would inform deployment planning and help organizations avoid the two failure modes: under-governed execution (too few humans for the task risk) and over-governed bottleneck (more human review than the throughput benefit justifies).

10. Protocol evolution and backward compatibility. The Skipjack Protocol defines a governance doctrine. As AI capabilities evolve — as the cognitive capability of agents in the Agentic Execution Layer changes, as new harness frameworks emerge, as the peer-reviewed understanding of governance effectiveness matures — the protocol will need to evolve. The research question is how a governance protocol updates without breaking existing governance contracts: organizations that have built compliance structures around Skipjack v1.0 need a versioning and migration path that preserves the governance integrity of existing deployments while incorporating improvements. Protocol evolution without backward compatibility is a governance risk; protocol stasis in a rapidly evolving capability environment is an equal and opposite risk.

8. CONCLUSION

The Skipjack Protocol described in this paper is a governing structure for the deployment of AI systems in which human judgment and agentic execution share operational responsibility. It

is not a replacement for the agent harness frameworks that provide the execution substrate. It is not a constraint on AI capability. It is a definition of what accountability looks like when that capability operates at scale.

The argument has been built from first principles. The gap between Agile Scrum's authority structure and the governance needs of agentic AI is not a coincidence — it is a structural parallel that justifies a direct mapping (§2). The Skipjack role remap is grounded in the peer-reviewed Scrum literature's established understanding of authority, accountability, and sprint discipline; it extends an existing governance tradition rather than inventing a new one. The four enforcement mechanisms (§3) are the operational form of that structure: context integrity that preserves intent through the execution chain, task decomposition that constrains scope before agents act, HITL control points that require human review at governance boundaries, and feedback loops that improve the system through governed iteration.

The context integrity manifest (§4) is not a novel claim of having solved provenance. The provenance infrastructure on which it builds — W3C PROV standards, workflow provenance specifications, the CWLProv implementation tradition — is well established. The manifest's contribution is applying that infrastructure to the governance of LLM context handoffs, treating provenance as a first-class governance artifact rather than an implementation detail. The original Governance Layer intent, the explicit exclusion record, the constraint registry in its unattenuated form, and the handoff receipt log together make intent traceable from Governance Layer to execution output. That traceability is what makes post-sprint reconciliation meaningful rather than performative.

The harness layer comparison in §6 locates the Skipjack Protocol in the production AI deployment landscape. The most capable multi-agent frameworks — AutoGen, MetaGPT, ChatDev — address the how of multi-agent coordination. The Skipjack Protocol addresses the under what authority. A Skipjack-compliant implementation could run on any of these substrates. The substrate provides the capability; the protocol defines the accountability structure within which that capability operates.

The future study vectors (§7) are an honest accounting of what remains open. The protocol has not been empirically validated at scale. The formal specification of its compliance conditions has not been written. The sector-specific compliance mappings have not been developed. Paper Nine and Paper Ten, the capstone implementation and results papers, will address the first of these gaps directly. The others are research programs, not corrections — the protocol is not incomplete because these questions are open. It is honest about the state of the field.

The Alistair Prime in a Box architecture (Paper Four) defines the unit. The Skipjack Protocol defines how humans and agents operate across the units. Both are expressions of the same underlying commitment: AI capability that serves the humans who deploy it, accountable to them throughout its operation, governable at every stage of the workflow.

The key to scaling AI is not autonomy alone — it is governed collaboration between human judgment and agentic execution.